

Lokalizacija zvuka mikrofonskim nizom

STM32G474 (akvizicija i DSP) + ESP32 (Wi-Fi 3D vizualizacija)

Ovaj dokument detaljno opisuje arhitekturu sustava, raspodjelu posla na FreeRTOS taskove, način procjene veličine stoga (stack), te cjelokupni lanac obrade signala — od mikrofona do kuta prikazanog u web-pregledniku. Pisan je tako da ga može razumjeti i osoba koja tek počinje s ugradbenim (embedded) sustavima i digitalnom obradom signala (DSP): svaki korak je objašnjen i popraćen konkretnim brojčanim primjerom.

Stavka	Vrijednost
Mikrokontroler (DSP)	STM32G474RE — Cortex-M4F @ 170 MHz, 128 KB RAM
Mikrofoni	4 analogna (elektret + pretpojačalo), niz u 3D rasporedu
Brzina uzorkovanja	64 kHz po kanalu (4 kanala, isti ADC)
Algoritam smjera	GCC-PHAT (TDOA) → geometrijsko 3D rješenje
Komunikacija	UART @ 115200 bps, binarni paketni protokol
Vizualizacija	ESP32 Wi-Fi Access Point + WebSocket + WebGL 3D
RTOS	FreeRTOS (CMSIS-OS v1 omotač) na STM32; FreeRTOS na ESP32

Tablica 1. Ključne značajke sustava.

Napomena o oznakama: kut azimuta dan je u rasponu $[-180^{\circ}, +180^{\circ}]$, a vrijednosti se prenose kao «desetinke stupnja» (tenths) — npr. 305 znači 30.5° .

Sadržaj

1. Pregled sustava — što i zašto
2. Arhitektura: dvije ploče, jedan tok podataka
3. Hardver i geometrija mikrofonskog niza
4. STM32 lanac akvizicije: TIM8 → ADC → DMA → ISR
5. FreeRTOS taskovi i protok podataka
6. Procjena veličine stoga (stack sizing)
7. Detekcija i lokalizacija zvuka — korak po korak
8. UART protokol između STM32 i ESP32
9. ESP32: Wi-Fi, web-poslužitelj i 3D vizualizacija
10. Cjeloviti primjer: od pljeska do točke na ekranu
11. Pojmovnik za početnike

1. Pregled sustava — što i zašto

Cilj sustava je **odrediti smjer iz kojeg dolazi zvuk** (npr. pljesak rukama) i taj smjer prikazati u stvarnom vremenu kao točku u 3D prostoru u web-pregledniku. Sustav ne prepoznaje *što* je zvuk, nego *odakle* dolazi — njegov azimut (kut u vodoravnoj ravnini) i elevaciju (kut prema gore).

Princip rada temelji se na jednostavnoj fizikalnoj činjenici: zvuk putuje konačnom brzinom (oko 343 m/s u zraku). Ako su mikrofoni razmaknuti u prostoru, isti zvučni val do njih stiže u **malo različitim trenucima**. Ta razlika u vremenu dolaska zove se **TDOA** (Time Difference Of Arrival). Iz nekoliko TDOA mjerenja i poznatih položaja mikrofona moguće je geometrijski izračunati smjer izvora.

Intuicija u jednoj rečenici

Ako mikrofoni M2 «čuje» pljesak 0.2 ms prije mikrofona M1, izvor je bliži mikrofoni M2 — a koliko točno «u stranu», govori nam omjer kašnjenja na svim parovima mikrofona.

Sustav je podijeljen na dvije ploče s jasno odvojenim ulogama:

- **STM32G474** — «mozak za obradu signala». Uzorkuje 4 mikrofona, obavlja svu matematiku (FFT, korelacija, geometrija) i kao rezultat šalje samo tri broja: azimut, elevaciju i jakost.
- **ESP32** — «prozor prema korisniku». Prima te brojeve preko UART-a, podiže vlastitu Wi-Fi mrežu i poslužuje web-stranicu s 3D prikazom. Ne radi nikakvu obradu zvuka.

Ovakva podjela je tipičan obrazac u ugradbenim sustavima: **mikrokontroler specijaliziran za determinističku obradu u stvarnom vremenu** (STM32 s FPU-om i DSP knjižnicom) odvojen je od **mikrokontrolera specijaliziranog za povezivost** (ESP32 s Wi-Fi/TCP-IP stogom). Svaki radi ono u čemu je dobar, a povezuje ih jednostavan serijski kabel.

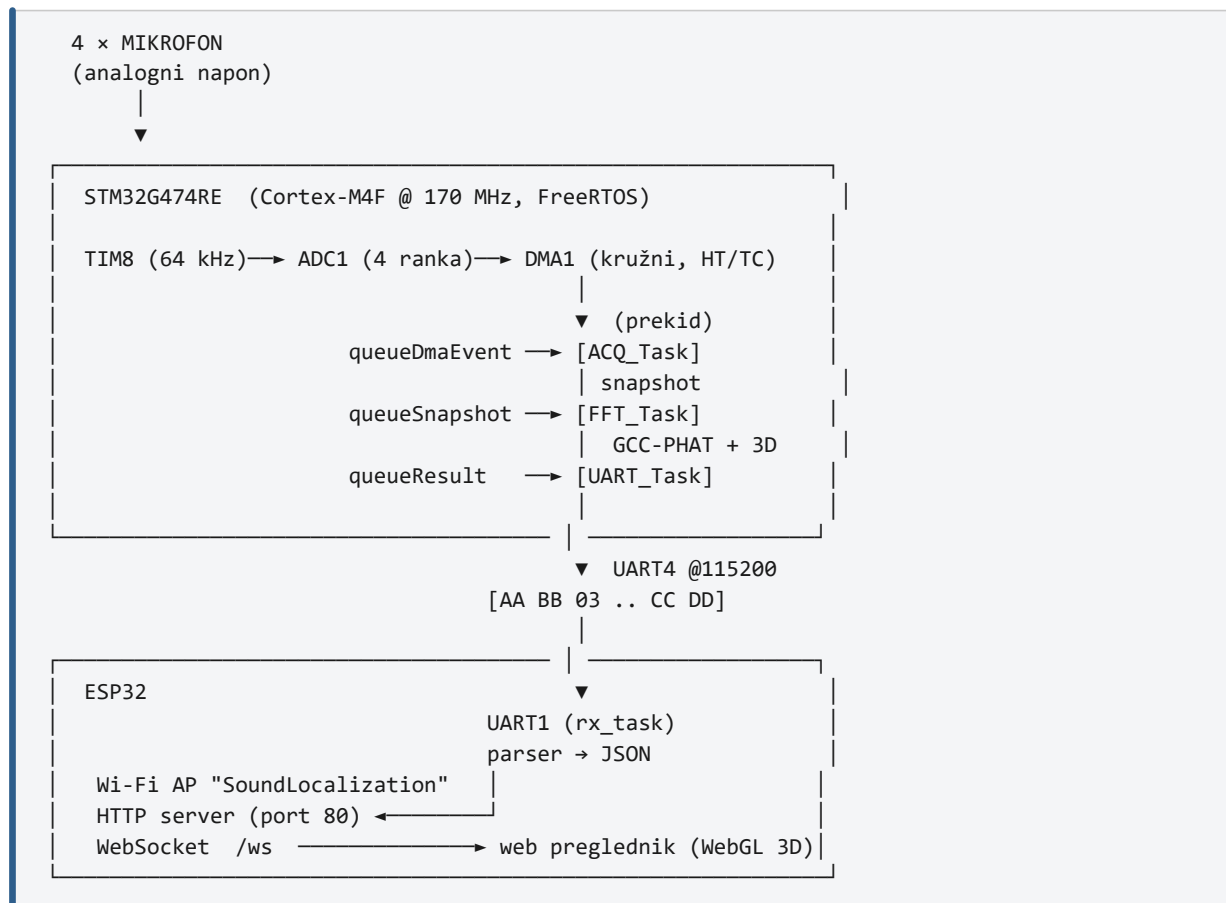
2. Arhitektura: dvije ploče, jedan tok podataka

Cijeli sustav najlakše je razumjeti kao **cjevovod** (pipeline): podatak ulazi kao zvučni tlak na mikrofoni, a izlazi kao piksel na ekranu. Na svakom koraku podatak mijenja oblik i postaje «sažetiji».

Korak	Oblik podatka	Količina
Zvučni val	tlak zraka	—
Mikrofon + pretpojačalo	analogni napon 0–3.3 V	4 kanala
ADC (TIM8 okida)	12-bitni uzorci (0–4095)	256 kS/s ukupno
DMA u RAM	interleaveani uint16 buffer	8192 uzoraka
DSP (FFT/GCC-PHAT)	kutovi azimut/elevacija	≈ 5 brojeva
UART paket	10 bajtova (tip 0x03)	po detekciji
ESP32 → WebSocket	JSON tekst	po detekciji
Web preglednik	3D točka (WebGL)	vizualno

Tablica 2. Tok podataka kroz sustav — svaki korak smanjuje količinu, a povećava «značenje» podatka.

2.1 Blok-shema



Slika 1. Blok-shema cijelog sustava. Lijevo-desno: fizički signal → digitalna obrada → mreža → ekran.

Primijetite tri **reda čekanja** (engl. *queue*) unutar STM32: oni razdvajaju dijelove sustava koji rade različitom brzinom. Prekidna rutina (ISR) i taskovi nikada ne dijele podatke izravno — uvijek preko reda čekanja. To je temelj sigurnog višezadačnog rada i objašnjeno je detaljno u poglavlju 5.

3. Hardver i geometrija mikrofonskog niza

Točnost lokalizacije izravno ovisi o tome koliko **precizno poznajemo položaje mikrofona**. Cijela matematika polazi od koordinata zadanih u datoteci `audio_common.h`. Mikrofon M1 je u ishodištu i služi kao **referentni** — svi se TDOA mjere u odnosu na njega.

Mikrofon	X [cm]	Y [cm]	Z [cm]	ADC rank	Pin
M1 (ref.)	0.00	0.00	0.00	RANK1	PB14 / IN5
M2	8.67	5.00	0.00	RANK2	PC0 / IN6
M3	8.67	-5.00	0.00	RANK3	PC1 / IN7
M4 (vrh)	5.00	0.00	8.00	RANK4	PC2 / IN8

Tablica 3. Položaji mikrofona i pridruženi ADC kanali. M1–M2–M3 čine ~jednakostranični trokut u ravnini, M4 je iznad baze.

Raspored je odabran promišljeno: tri mikrofona (M1, M2, M3) leže u vodoravnoj ravnini i daju **azimut**, dok je četvrti (M4) podignut u visinu ($Z = 8$ cm) i tek on omogućuje mjerenje **elevacije**. Bez mikrofona izvan ravnine sustav ne bi mogao razlikovati zvuk odozgo od zvuka odozdo.

3.1 Koordinatni sustav i značenje kutova

- **+X** → 0° azimut: «naprijed» (simetrala između M2 i M3).
- **+Y** → $+90^\circ$ azimut: lijevo (strana mikrofona M2).

- **-Y** → -90° azimut: desno (strana mikrofona M3).
- **-X** → ±180° azimut: nazad (strana mikrofona M1).
- **+Z** → +90° elevacija: ravno gore (smjer mikrofona M4).

Zašto baš ~10 cm razmaka?

Razmak određuje dvije suprotstavljene stvari. Veći razmak → veće (lakše mjerljivo) kašnjenje → bolja kutna rezolucija. Ali ako je razmak veći od pola valne duljine najviših frekvencija, javlja se prostorni «aliasing» (dvosmislenost smjera). Razmak od ~10 cm pri brzini zvuka 343 m/s daje najveće kašnjenje od oko 18.7 uzoraka na 64 kHz — odatle konstanta `TDOA_MAX_SAMPLES = 20` koja ograničava pretragu vrha korelacije.

Računica gornje granice (zašto 20): najveći razmak od referentnog mikrofona je ~10 cm. Vrijeme da zvuk prijeđe 10 cm iznosi $0.10 \text{ m} \div 343 \text{ m/s} = 291.5 \text{ } \mu\text{s}$. Pri 64 000 uzoraka u sekundi to je $291.5 \text{ } \mu\text{s} \times 64\,000 = \mathbf{18.66 \text{ uzoraka}}$. Zaokruženo naviše i s malom rezervom → 20.

4. STM32 lanac akvizicije: TIM8 → ADC → DMA → ISR

Prije bilo kakve matematike treba **pretvoriti zvuk u niz brojeva** pouzdanom, ravnomjernom brzinom. Ovaj lanac to radi gotovo bez sudjelovanja procesora — hardver sam «puni» memoriju, a procesor se javlja tek kad je blok podataka spreman. To je ključ za rad u stvarnom vremenu.

4.1 TIM8 — generator takta uzorkovanja

Tajmer TIM8 postavljen je da se preljeva (overflow) točno 64 000 puta u sekundi. Pri svakom preljevu šalje interni okidač (TRGO) prema ADC-u. Time je **brzina uzorkovanja potpuno hardverska i jednolika** — ne ovisi o tome koliko je procesor zauzet.

```
TIM_InitStruct.Prescaler = 0;
TIM_InitStruct.Autoreload = 2655;      /* ARR */
LL_TIM_SetTriggerOutput(TIM8, LL_TIM_TRGO_UPDATE); /* TRGO na svaki update */

/* Frekvencija = f_tim / (ARR + 1) = 170 MHz / (2655 + 1)
   = 170 000 000 / 2656 = 64 006 Hz ≈ 64 kHz
   Perioda = 1 / 64000 = 15.625 μs po uzorku */
```

Isječak 1. Konfiguracija TIM8 (timer_driver.c / main.c). ARR = 2655 daje ≈ 64 kHz.

4.2 ADC — jedan pretvarač, četiri kanala redom

Sustav koristi **jedan** ADC (ADC1) koji u «scan» načinu pretvara četiri kanala jedan za drugim pri svakom okidaču tajmera. Redoslijed (rank) određuje i raspored podataka u memoriji.

Bitan detalj koji početnici lako previde: budući da kanali nisu uzorkovani **istovremeno** nego redom, svaki sljedeći kanal kasni za prethodnim za vrijeme jedne konverzije. To unosi sustavnu pogrešku u TDOA koju kod kasnije ispravlja (vidi 7.6).

```
/* ADC takt = PCLK / 4 = 170 MHz / 4 = 42.5 MHz → perioda 23.53 ns
   Po kanalu: 2.5 (uzorkovanje) + 12.5 (konverzija) = 15 ciklusa
   = 15 × 23.53 ns = 352.9 ns */
#define CH_DELAY_S 352.9e-9f

/* Posljedica sekvencijalnog čitanja:
   M1 uzorkovan u t + 0×352.9 ns
   M2 uzorkovan u t + 1×352.9 ns
   M3 uzorkovan u t + 2×352.9 ns
   M4 uzorkovan u t + 3×352.9 ns */
```

Isječak 2. Kanalni offset zbog sekvencijalne konverzije (audio_common.h).

4.3 DMA — hardver puni memoriju umjesto procesora

DMA (Direct Memory Access) je sklop koji premješta svaki rezultat ADC-a izravno u RAM **bez prekidanja procesora za svaki uzorak**. Konfiguriran je kao **kružni** (circular): kad napuni kraj buffera, automatski se vrati na početak. Buffer je dvostruke veličine (8192 uzoraka) i koristi se kao **dvostruki spremnik** (double buffer):

- Dok DMA puni **drugu** polovicu buffera, procesor smije čitati **prvu** polovicu — i obrnuto. Nikad ne pišu i ne čitaju isti dio.
- DMA javlja dva prekida: **HT** (Half Transfer, popunjena prva polovica) i **TC** (Transfer Complete, popunjena druga polovica).

```
#define SAMPLES_PER_CHANNEL 1024
#define NUM_CH 4
#define HALF_BUFFER (NUM_CH * SAMPLES_PER_CHANNEL) /* 4096 uzoraka */
#define FULL_BUFFER (HALF_BUFFER * 2) /* 8192 uzoraka */

uint16_t adc_buffer[FULL_BUFFER]; /* 8192 x 2 B = 16 KB u RAM-u */

/* Raspored u memoriji je INTERLEAVED (kanali isprepleteni):
   adc_buffer[s*4 + 0] = M1    adc_buffer[s*4 + 2] = M3
   adc_buffer[s*4 + 1] = M2    adc_buffer[s*4 + 3] = M4 */
```

Isječak 3. Buffer i njegov interleavedni raspored (adc_driver.c, audio_common.h).

Jedna «polovica» = 4096 uzoraka = 1024 vremenska trenutka × 4 kanala. Pri 64 kHz, 1024 uzorka po kanalu traje $1024 \div 64000 = 16$ ms. Dakle svakih 16 ms stigne jedan HT ili TC prekid — to je «otkucaj srca» cijelog sustava.

4.4 ISR — kratak prekid koji samo javlja «gotovo»

Prekidna rutina mora biti **što kraća**. Ova ne radi nikakvu obradu — samo pošalje broj (0 za HT, 1 za TC) u red čekanja i, ako je time probudila task višeg prioriteta, zatraži preraspodjelu procesora. Sva «teška» obrada događa se izvan prekida, u tasku.

```
void DMA1_Channel1_IRQHandler(void) {
    BaseType_t woken = pdFALSE;
    uint32_t msg;
    if (LL_DMA_IsActiveFlag_HT1(DMA1)) { /* prva polovica gotova */
        LL_DMA_ClearFlag_HT1(DMA1);
        msg = 0u;
        xQueueSendFromISR(queueDmaEventHandle, &msg, &woken);
    }
    if (LL_DMA_IsActiveFlag_TC1(DMA1)) { /* druga polovica gotova */
        LL_DMA_ClearFlag_TC1(DMA1);
        msg = 1u;
        xQueueSendFromISR(queueDmaEventHandle, &msg, &woken);
    }
    portYIELD_FROM_ISR(woken); /* probudi ACQ_Task ako treba */
}
```

Isječak 4. DMA prekidna rutina (stm32g4xx_it.c). Pravilo: u ISR-u radi minimum, ostalo prepusti tasku.

5. FreeRTOS taskovi i protok podataka

FreeRTOS je **operativni sustav za stvarno vrijeme** (RTOS). Omogućuje da se posao podijeli na neovisne «niti» (taskove) koje scheduler izmjenjuje na procesoru prema **prioritetu**. Svaki task ima vlastiti stog (stack) i vrti se u beskonačnoj petlji, najčešće čekajući na nekom redu čekanja.

Sustav koristi tri taska, povezana s tri reda čekanja, po načelu **proizvođač–potrošač**: svaki task uzima posao iz svog ulaznog reda, obradi ga i rezultat preda u sljedeći red.

Task	Prioritet	Stog	Uloga
ACQ_Task	Realtime (najviši)	256 riječi	Kopira svjež u polovicu buffera u snapshot
FFT_Task	High	1024 riječi	GCC-PHAT + 3D geometrija (sva matematika)
UART_Task	Low (najniži)	256 riječi	Šalje rezultat (i sirove uzorke) na ESP32

Tablica 4. Tri taska, njihovi prioriteti i stogovi (*task_manager.c*, *app_tasks_init*).

Logika prioriteta: **akvizicija je najvažnija** jer ako propustimo svjež podatak iz DMA-a, izgubili smo ga zauvijek. Obrada (FFT) je važna ali smije malo pričekati. Slanje na UART je najmanje hitno (ESP32 prikaz toleriran je kasni desetke milisekundi), pa ima najniži prioritet.

5.1 Tri reda čekanja

```
queueDmaEventHandle : DMA ISR → ACQ_Task (dubina 8, uint32 događaj HT/TC)
queueSnapshotHandle : ACQ_Task → FFT_Task (dubina 2, indeks buffera uint8)
queueResultHandle   : FFT_Task → UART_Task (dubina 4, loc3d_result_t)
```

Isječak 5. Tri reda čekanja povezuju ISR i tri taska (*task_manager.h / .c*).

5.2 ACQ_Task — siguran prijenos svježe polovice

ACQ_Task čeka na događaj iz DMA prekida. Kad dobije 0 (HT) zna da je gotova prva polovica buffera, kad dobije 1 (TC) druga. Tu polovicu kopira u jedan od tri «snapshot» buffera i indeks pošalje dalje. Zašto tri buffera i kopiranje? Da DMA (koji nikad ne staje) ne prepíše podatke dok ih FFT_Task još obrađuje.

```
static void StartACQTask(void const *argument) {
    uint8_t write_idx = 0;
    for (;;) {
        osEvent evt = osMessageGet(queueDmaEventHandle, osWaitForever);
        if (evt.status != osEventMessage) continue;

        /* Ako FFT_Task zaostaje i red je pun – preskoči, ne prepisuj buffer
           koji se možda još čita (sprječava «torn read»). */
        if (uxQueueSpacesAvailable(queueSnapshotHandle) == 0u) continue;

        const uint16_t *src = (evt.value.v == 0u) ? &adc_buffer[0]
                                                : &adc_buffer[HALF_BUFFER];
        memcpy(acq_snapshot[write_idx], src, HALF_BUFFER * sizeof(uint16_t));

        xQueueSend(queueSnapshotHandle, &write_idx, 0u);
        write_idx = (write_idx + 1u) % ACQ_NUM_BUFFERS; /* 0→1→2→0 */
    }
}
```

Isječak 6. ACQ_Task (*task_manager.c*). Trostruki spremnik + provjera praznine reda = nema oštećenih podataka.

Pojam: torn read (poderano čitanje)

Ako jedan task čita niz dok ga drugi (ili DMA) istovremeno mijenja, dobije mješavinu starih i novih vrijednosti — besmislen podatak. Rješenje ovdje: ACQ kopira podatak u zaseban buffer i nikad ne prepisuje onaj koji je još «u opticaju» (u redu čekanja ili u obradi).

5.3 FFT_Task — klizni prozor i sva matematika

FFT_Task uzima indeks svježe polovice i nadovezuje je na prethodnu, tvoreći **klizni prozor** (sliding window) od $2 \times 1024 = 2048$ vremenskih trenutaka. Razlog: pljesak može pasti na granicu dviju polovica; s dvostrukim prozorom uvijek imamo cijeli događaj na jednom mjestu, pa ga algoritam može «centrirati».

```
static void StartFFTTask(void const *argument) {
    loc3d_result_t result;
    uint8_t read_idx;
    for (;;) {
        if (xQueueReceive(queueSnapshotHandle, &read_idx, portMAX_DELAY) != pdTRUE)
            continue;

        /* pomakni stari «desni» pola u «lijevi», pa dodaj novu polovicu desno */
        memcpy(&sliding_buf[0], &sliding_buf[HALF_BUFFER],
            HALF_BUFFER*sizeof(uint16_t));
        memcpy(&sliding_buf[HALF_BUFFER], acq_snapshot[read_idx],
            HALF_BUFFER*sizeof(uint16_t));

        if (!capture_ready) {
            if (LOC3D_Process(sliding_buf, &result)) { /* ← sva obrada ovdje */
                capture_ready = 1;
                xQueueSend(queueResultHandle, &result, 0);
            }
        }
    }
}
```

Isječak 7. FFT_Task (task_manager.c). LOC3D_Process je srce DSP-a — razrađeno u poglavlju 7.

5.4 UART_Task i mehanizam «capture_ready»

UART_Task čeka rezultat i šalje ga na ESP32. Dodatno, nakon kuta opcionalno šalje i **sirove uzorke** (8 KB) za analizu u MATLAB-u. Slanje 8 KB na 115200 bps traje ~0.7 s — pravu vječnost za sustav koji okida svakih 16 ms. Zato zastavica capture_ready «zamrzne» obradu dok traje slanje, da se sirovi buffer ne prepíše usred prijenosa.

```
static void StartUARTTask(void const *argument) {
    loc3d_result_t r;
    for (;;) {
        if (xQueueReceive(queueResultHandle, &r, portMAX_DELAY) == pdTRUE) {
            UART_SendAngle3DPacket(r.az_tenth, r.el_tenth, r.strength);
            if (capture_ready) {
                UART_SendRawCapture(dbg_raw_ch0, dbg_raw_ch1,
                    dbg_raw_ch2, dbg_raw_ch3);
                capture_ready = 0; /* obrada se nastavlja */
            }
        }
    }
}
```

Isječak 8. UART_Task (task_manager.c). capture_ready je jednostavan «handshake» između FFT i UART taska.

6. Procjena veličine stoga (stack sizing)

Svaki FreeRTOS task ima **vlastiti stog** — područje RAM-a na koje se spremaju lokalne varijable, argumenti funkcija i adrese povratka pri svakom ugnjeđenom pozivu. Ako task potroši više od dodijeljenog stoga, dolazi do **prelijevanja stoga** (stack overflow) — jedne od najčešćih i najpodmuklijih grešaka u ugradbenim sustavima, jer se očituje kao naizgled slučajno rušenje.

Jedinica: «riječ» (word), ne bajt

U FreeRTOS-u se veličina stoga zadaje u **riječima**. Na Cortex-M4 jedna riječ = 4 bajta. Dakle 256 riječi = 1024 bajta = 1 KB, a 1024 riječi = 4 KB.

6.1 Što sve troši stog

- **Lokalne varijable** svake funkcije u lancu poziva (najveći potrošač su velika lokalna polja).
- **Spremanje registara** pri pozivu funkcije i, na Cortex-M4F, **FPU registara** kod operacija s pomičnim zarezom.
- **Ugnježđivanje poziva** — svaki dublji poziv dodaje novi «okvir».
- **Kontekst prekida** — prekid koji se dogodi dok task radi nakratko koristi stog (ovisno o portu).

6.2 Kako se procjenjuje u praksi

Procjena ide kombinacijom **analize** i **mjerenja**. Ključ je shvatiti da **velika polja podataka NE žive na stogu** u ovom projektu — namjerno su deklarirana kao `static` (u BSS segmentu), pa ne opterećuju stog taska. To je svjesna projektna odluka koja drži stogove malima.

```
/* Ova polja su 'static' → u BSS-u, NE na stogu FFT_Task-a: */
static float s_ch0[1024], s_ch1[1024], s_ch2[1024], s_ch3[1024]; /* 16 KB */
static float s_corr[1024]; /* 4 KB */

/* Unutar GCC_PHAT — također 'static', dijele se kroz pozive: */
static float fft_x[1024], fft_y[1024], fft_c[1024]; /* 12 KB */
```

Isječak 9. Veliki radni nizovi su 'static' (*sound_loc_3d.c*, *gcc_phat.c*) — zato *FFT_Task* treba samo ~4 KB stoga.

Da su ti nizovi bili obične lokalne varijable, samo jedan poziv `LOC3D_Process` tražio bi preko 30 KB stoga — višestruko više od dodijeljenog. Ovako na stogu ostaju samo skalari (pokazivači, brojači petlji, međurezultati tipa `float`).

Postupak u tri koraka

- 1 **Analitička gornja procjena.** Prođi kroz najdublji lanac poziva taska i zbroji lokalne varijable + rezervu za registre/FPU/prekid. Za *FFT_Task* najdublji put je `LOC3D_Process` → `GCC_PHAT` → `arm_rfft_fast_f32`; budući da su veliki nizovi `static`, ostaje par stotina bajta skalara + CMSIS-DSP interni okviri. S velikodušnom rezervom → 1024 riječi (4 KB).
- 2 **Mjerenje u radu (high-water mark).** FreeRTOS nudi `uxTaskGetStackHighWaterMark()` koji vraća *najmanju* količinu slobodnog stoga ikad zabilježenu za taj task. Pustiš sustav da radi (puno pljeskanja, rubni slučajevi), pa očitaš koliko je stoga stvarno ostalo neiskorišteno.
- 3 **Sigurnosna rezerva.** Ostavi barem 25–50% slobodnog stoga iznad izmjerenog vrha, za nepredviđene dublje putove (npr. drugačiji ulaz koji aktivira drugu granu koda).

6.3 Mreža sigurnosti: dvije zaštite

Procjena nikad nije savršena, pa kod ima dvije aktivne zaštite:

```
/* FreeRTOSConfig.h */
#define configENABLE_FPU 1 /* čuva FPU registre pri context switchu —
                           nužno jer FFT_Task koristi float/asinf */
#define configCHECK_FOR_STACK_OVERFLOW ... /* (preko hook-a niže) */

/* main.c — poziva se automatski ako task prijeđe svoj stog: */
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName) {
    __disable_irq();
    while (1) { } /* zaustavi sve — lako uočljivo u debuggeru */
}
```

Isječak 10. Zaštite vezane uz stog (*FreeRTOSConfig.h*, *main.c*).

Zašto je `configENABLE_FPU = 1` obavezan ovdje

FFT_Task intenzivno koristi pomični zarez (*asinf*, *atan2f*, množenja matrice). Ako scheduler prekine task usred float-operacije, a druga nit dotakne FPU, bez čuvanja FPU konteksta vrijednosti se nepovratno pokvare. Uz `FPU=1` FreeRTOS koristi «lijeno» (*lazy*) spremanje FPU registara — čuva ih samo kad task stvarno koristi FPU, štedeći stog i vrijeme.

6.4 Heap (gomila) cijelog RTOS-a

Osim stogova pojedinih taskova, FreeRTOS ima i zajednički **heap** iz kojeg se alociraju stogovi taskova, redovi čekanja i drugi objekti. Postavljen je na 24 KB, što je s rezervom dovoljno:

Objekt	Procjena
ACQ_Task stog (256 riječi)	1.0 KB
FFT_Task stog (1024 riječi)	4.0 KB
UART_Task stog (256 riječi)	1.0 KB
3 reda čekanja + RTOS overhead	≈ 0.6 KB
Ukupno zauzeto	≈ 6.6 KB
<code>configTOTAL_HEAP_SIZE</code>	24 KB
Slobodno za budući rast	≈ 17 KB

Tablica 5. Bilanca FreeRTOS heap-a (*FreeRTOSConfig.h*). STM32G474RE ima 128 KB RAM-a, pa je 24 KB tek ~19%.

Napomena: veliki nizovi (*adc_buffer* 16 KB, *snapshoti* 24 KB, *klizni prozor* 16 KB, *DSP nizovi* ~32 KB) **ne dolaze iz heap-a** — oni su statički globalni (BSS) i linker ih smješta zasebno. Heap služi isključivo RTOS objektima.

7. Detekcija i lokalizacija zvuka — korak po korak

Ovo je srž projekta. Cijeli postupak izvodi funkcija `LOC3D_Process()`, koju `FFT_Task` zove jednom za svaku novu polovicu buffera (svakih 16 ms). Niže je svaki korak objašnjen i popraćen **brojčanim primjerom** za jedan zamišljeni pljesak.

Postavka primjera koji pratimo kroz cijelo poglavlje

Pretpostavimo da pljesak dolazi iz smjera azimut +30°, elevacija +20°, dovoljno glasan. Kroz korake ćemo vidjeti kako iz sirovih uzoraka ispadnu upravo ti kutovi.

7.1 Korak 0 — Cooldown (sprječavanje višestrukih okidanja)

Jedan pljesak traje i odzvanja nekoliko desetaka milisekundi i prelijeva se preko više prozora. Bez kočnice sustav bi za jedan pljesak ispalio 5–10 detekcija. Zato nakon svake uspješne detekcije slijedi «hlađenje» od 19 prozora (≈ 304 ms) tijekom kojih se obrada preskače.

```
#define DETECTION_COOLDOWN_FRAMES 19 /* 19 × 16 ms ≈ 304 ms */
static uint16_t s_cooldown = 0;

if (s_cooldown > 0) { s_cooldown--; return 0; } /* preskoči, jeftino */
```

Isječak 11. Cooldown (*sound_loc_3d.c*). Primjer: detekcija u $t=0 \rightarrow$ sljedeća moguća tek nakon ~304 ms.

7.2 Korak 1 — Pronalazak energetskog vrha (gdje je pljesak)

Pljesak je kratak, a prozor traje 32 ms. Algoritam prvo pronađe gdje **unutar** kliznog prozora leži najglasniji dio i ondje centrira analizu. Koristi se **klizeći prozor energije** širine 16 uzoraka koji se inkrementalno pomiče — umjesto da za svaki položaj iznova zbraja, oduzme uzorak koji «izlazi» i doda onaj koji «ulazi». To je $O(N)$ umjesto $O(N \times W)$.

```
/* energija = suma kvadrata (uzorak - 2048), 2048 = sredina 12-bitnog raspona */
int32_t v = (int32_t)s[ch] - 2048;
win_e += (v * v) >> 6;          /* >>6 sprječava preljev akumulatora */

/* klizni pomak: skini stari rub, dodaj novi */
win_e -= (v_old*v_old) >> 6;
win_e += (v_new*v_new) >> 6;
if (win_e > best_e) { best_e = win_e; best_frame = f + PROBE/2; }
```

Isječak 12. *find_peak_offset (sound_loc_3d.c)*. Rezultat: *frame_offset* = početak 1024-uzorčanog prozora centriranog na pljesak.

Primjer 7.2

Recimo da je pljesak najglasniji oko 1200-tog trenutka u kliznom prozoru (0–2047). Algoritam vrati *frame_offset* $\approx 1200 - 512 = 688$, tj. analizirat će uzorke [688 ... 1711] gdje je događaj lijepo u sredini.

7.3 Korak 2 — Deinterleave, uklanjanje DC-a i Hann prozor

Iz interleaveanog buffera treba izdvojiti **4 zasebna kanala**. Pritom se obavljaju dvije pripreme nužne za korelaciju:

- **Uklanjanje istosmjerne komponente (DC)**. Mikrofon «sjedi» na nekom naponu (npr. $\sim 1.65\text{ V} \rightarrow \text{ADC} \sim 2048$). Za korelaciju nas zanima samo *promjena* oko te sredine, pa od svakog uzorka oduzmemo prosjek kanala.
- **Hann prozor**. Množenje signala glatkim «zvonom» (na rubovima $\rightarrow 0$) smanjuje artefakte FFT-a (spektralno curenje) koji nastaju jer FFT pretpostavlja da je signal periodičan.

```
float w = s_hann[s];          /* unaprijed izračunato zvono */
ch0[s] = ((float)r0 - dc0) * w; /* (uzorak - DC) × prozor */
...
/* Hann: w[i] = 0.5 · (1 - cos(2π·i / (N-1))), N = 1024 */
```

Isječak 13. *GCC_ExtractChannels (gcc_phat.c)*. Pretvara *uint16* $\rightarrow$ *float*, makne DC, primijeni Hann.

Primjer 7.3

Uzorak *M1* = 2100, prosjek kanala *dc0* = 2048, a Hann na tom mjestu *w* = 0.95. Rezultat: $(2100 - 2048) \times 0.95 = 52 \times 0.95 = 49.4$. Tako «očišćen» signal ide u FFT.

7.4 Korak 3 — RMS prag (je li uopće bilo dovoljno glasno)

RMS (Root Mean Square) je mjera «glasnoće» — efektivna amplituda signala. Ako je najglasniji kanal tiši od praga, riječ je o tišini/šumu i obrada se prekida. Drugi (blaži) prag traži da ni najtiši kanal ne bude posve mrtav, čime se odbacuju situacije kad samo jedan mikrofon nešto uhvati.

```
#define MIN_RMS_THRESHOLD    10.0f
#define MIN_RMS_PER_CHANNEL  (MIN_RMS_THRESHOLD * 0.2f) /* = 2.0 */

if (rms_max < MIN_RMS_THRESHOLD) return 0; /* pretiho - izađi */
if (rms_min < MIN_RMS_PER_CHANNEL) return 0; /* jedan mik mrtav */
```

Isječak 14. *RMS prag (sound_loc_3d.c)*. $\text{RMS} = \sqrt{\text{prosjeck kvadrata uzoraka}}$.

Primjer 7.4

Pljesak: $\text{rms} = [42, 38, 35, 30]$. $\text{rms_max} = 42 > 10$ i $\text{rms_min} = 30 > 2 \rightarrow$ prolazi. Tiha soba: $\text{rms} = [3, 2, 3, 2] \rightarrow \text{rms_max} = 3 < 10 \rightarrow$ odmah izlaz.

7.5 Korak 4 — GCC-PHAT: mjerenje kašnjenja između parova

Ovo je matematička jezgra. Za svaki par (M1–M2, M1–M3, M1–M4) treba izmjeriti **za koliko uzoraka jedan signal kasni za drugim**. Naivni pristup je **križna korelacija**: kliži jedan signal preko drugog i traži pomak pri kojem se najbolje poklapaju. GCC-PHAT je njena «izoštrena» inačica.

Zašto PHAT, a ne obična korelacija

Obična korelacija daje širok, zaobljen vrh — teško je točno odrediti gdje je maksimum, osobito uz jeku i šum. **PHAT** (Phase Transform) u frekvencijskoj domeni **odbaci informaciju o amplitudi i zadrži samo fazu** svake frekvencije. Rezultat je vrlo oštar, gotovo «šiljast» vrh točno na pravom kašnjenju — jer kašnjenje je u biti linearni nagib faze po frekvenciji.

Postupak po paru izvodi se preko brzog FFT-a (CMSIS-DSP `arm_rfft_fast_f32`):

- 1 Izračunaj FFT oba signala: $X = \text{FFT}(\text{ref})$, $Y = \text{FFT}(\text{sig})$.
- 2 Za svaku frekvenciju izračunaj **križni spektar** $C = \text{konj}(X) \cdot Y$ (konj = kompleksno konjugiranje).
- 3 **PHAT normalizacija**: podijeli svaki C s njegovim iznosom $|C|$, tako da ostane samo faza (jedinичni iznos).
- 4 Inverzni FFT od normaliziranog $C \rightarrow$ korelacijska sekvenca u vremenu. Položaj njenog vrha je traženo kašnjenje.

```
arm_rfft_fast_f32(&s_rfft, ref, fft_x, 0); /* X = FFT(ref) */
arm_rfft_fast_f32(&s_rfft, sig, fft_y, 0); /* Y = FFT(sig) */

/* za svaki frekvencijski bin k: */
float cre = re1*re2 + im1*im2; /* Re{ konj(X)·Y } */
float cim = re1*im2 - im1*re2; /* Im{ konj(X)·Y } */
float mag = sqrtf(cre*cre + cim*cim);
float w = (mag > 1e-9f) ? 1.0f/(mag + 1e-9f) : 0.0f; /* PHAT: 1/|C| */
fft_c[iire] = cre*w; fft_c[iim] = cim*w;

arm_rfft_fast_f32(&s_rfft, fft_c, corr, 1); /* IFFT → korelacija */
```

Isječak 15. `GCC_PHAT (gcc_phat.c)`. $\text{Konj}(X) \cdot Y$ daje vrh na pozitivnom kašnjenju kad *sig* kasni za *ref*.

Primjer 7.5 (predznak govori smjer)

Za par M1–M2: ako vrh korelacije ispadne na lagu +5 uzoraka, to znači da M2 kasni 5 uzoraka za M1 $\rightarrow$ val je do M1 stigao prije $\rightarrow$ izvor je bliži M1-strani. Predznak i iznos svih triju kašnjenja zajedno jednoznačno kodiraju smjer.

7.6 Korak 5 — Točan vrh: pretraga + parabolička interpolacija

Korelacija je uzorkovana u cijelim uzorcima, ali pravo kašnjenje rijetko pada točno na uzorak. Zato se nakon pronalaska najvišeg uzorka radi **parabolička interpolacija**: kroz vrh i dva susjeda provuče se parabola i njen tjeme daje **pod-uzorčano** (sub-sample) kašnjenje. Pretraga je ograničena na ± 20 uzoraka (fizička granica iz poglavlja 3).

```

/* najveći APSOLUTNI vrh u rasponu ±TDOA_MAX_SAMPLES (hvata i negativan
   polaritet ako je neki mikrofon obrnuto spojen) */
delta = 0.5f * (c0 - c2) / (c0 - 2*c1 + c2); /* tjeme parabole */
float frac = (float)max_idx + delta; /* npr. 5 + 0.3 = 5.3 */
return delay_samp * SAMPLE_PERIOD_S; /* uzorci → sekunde */

/* Usput: 'qual' = vrh / prosjek|korelacije| – visok za pravi signal,
   ~1 za šum. Trenutno je prag kvalitete 0 (gate isključen radi ugađanja).*/

```

Isječak 16. GCC_FindTDOA (gcc_phat.c). Pretvara položaj vrha u kašnjenje u sekundama.

Korekcija sekvencijalnog ADC offseta

Sjetimo se da kanali nisu uzorkovani istovremeno (poglavlje 4.2). GCC-PHAT izmjeri **pravidno** kašnjenje koje uključuje i taj hardverski pomak. Stvarno **akustičko** kašnjenje dobije se dodavanjem $(j-1) \times \text{CH_DELAY}$:

```

float tau12 = tau12_meas + 1.0f * CH_DELAY_S; /* M2 = 2. rank */
float tau13 = tau13_meas + 2.0f * CH_DELAY_S; /* M3 = 3. rank */
float tau14 = tau14_meas + 3.0f * CH_DELAY_S; /* M4 = 4. rank */

```

Isječak 17. Korekcija kanalnog offseta (sound_loc_3d.c).

Primjer 7.6 (naši brojevi)

Za smjer $+30^\circ/+20^\circ$ prava akustička kašnjenja izračunata iz geometrije su: $\tau_{12} \approx -274 \mu\text{s}$ (-17.5 uzoraka), $\tau_{13} \approx -137 \mu\text{s}$ (-8.8), $\tau_{14} \approx -198 \mu\text{s}$ (-12.7). GCC-PHAT izmjeri približno te vrijednosti umanjene za kanalni pomak; korak korekcije ih vrati na navedene «prave» iznose.

7.7 Korak 6 — Geometrijsko rješenje: od kašnjenja do smjera

Tri kašnjenja (τ_{12} , τ_{13} , τ_{14}) i poznata geometrija jednoznačno određuju smjer. Veza je linearna: ako složimo baseline vektore u matricu **D** ($\text{reci} = \text{Mj} - \text{M1}$), vrijedi $\mathbf{D} \cdot \mathbf{s} = -\mathbf{c} \cdot \boldsymbol{\tau}$, gdje je **s** jedinični vektor prema izvoru, a **c** brzina zvuka.

Rješavanje po **s** svodi se na množenje unaprijed izračunatom matricom $\text{M_geom} = \mathbf{c} \cdot \text{inv}(\mathbf{D})$. Inverzija 3×3 računa se **jednom** pri pokretanju (LOC3D_Init), pa je po detekciji potrebno samo 9 množenja i 6 zbrajanja — vrlo jeftino.

```

/* jednom na startu: */
M_geom = c * inv(D), D = [ M2-M1 ; M3-M1 ; M4-M1 ] (3x3)

/* po detekciji: u = smjer propagacije = M_geom * tau */
ux = M_geom[0][0]*tau12 + M_geom[0][1]*tau13 + M_geom[0][2]*tau14;
uy = M_geom[1][0]*tau12 + M_geom[1][1]*tau13 + M_geom[1][2]*tau14;
uz = M_geom[2][0]*tau12 + M_geom[2][1]*tau13 + M_geom[2][2]*tau14;

/* smjer PREMA izvoru je suprotan smjeru propagacije: s = -u, pa normaliziraj */
sx = -ux; sy = -uy; sz = -uz; norm = sqrt(sx^2+sy^2+sz^2); s /= norm;

```

Isječak 18. Geometrijsko rješenje (LOC3D_Init + LOC3D_Process, sound_loc_3d.c).

Zašto inverzija samo jednom

Matrica **D** ovisi isključivo o položajima mikrofona, koji se ne mijenjaju u radu. Skupu operaciju (inverziju) napravimo jednom i spremimo rezultat; u vrućoj petlji ostaje samo jeftino množenje matrice **s** vektorom. Tipičan DSP obrazac: «pretporačunaj sve što je konstantno».

7.8 Korak 7 — Kutovi azimut i elevacija

Iz jediničnog vektora smjera $s = (s_x, s_y, s_z)$ kutovi se dobiju izravno trigonometrijom. Rezultat se sprema u desetinkama stupnja (cijeli broj) radi kompaktnog prijenosa.

```
float az_rad = atan2f(sy, sx);    /* azimut: [-180°, +180°] */
float el_rad = asinf(sz);        /* elevacija: [-90°, +90°] */

out->az_tenth = (int16_t)(az_rad * 1800.0f / PI); /* radijani → 0.1° */
out->el_tenth = (int16_t)(el_rad * 1800.0f / PI);
```

Isječak 19. Pretvorba vektora u kutove (sound_loc_3d.c).

Primjer 7.8 (zatvaranje kruga)

Za naš pljesak geometrija vrati $s \approx (0.814, 0.470, 0.342)$. Tada je $\text{atan2}(0.470, 0.814) = 30.0^\circ \rightarrow \text{az_tenth} = 300$, a $\text{asin}(0.342) = 20.0^\circ \rightarrow \text{el_tenth} = 200$. Točno smjer iz kojeg smo «pustili» pljesak na početku poglavlja.

7.9 Korak 8 — Jakost signala

Za vizualizaciju je korisna i «jakost» 0–100. Računa se logaritamski (slično decibelima) jer ljudski doživljaj glasnoće i raspon pljeskova ($\text{RMS} \approx 30\text{--}500$) najbolje opisuje log-ljestvica.

```
float str_f = 20.0f * log10f(rms_max + 1.0f); /* dB-slično */
if (str_f < 1.0f) str_f = 1.0f;
if (str_f > 100.0f) str_f = 100.0f;
out->strength = (uint8_t)str_f;
/* RMS 20 → ~26, RMS 100 → ~40, RMS 500 → ~54 */
```

Isječak 20. Izračun jakosti (sound_loc_3d.c).

Na kraju, ako je sve prošlo, postavi se cooldown i funkcija vrati 1 (valjana detekcija). FFT_Task tada rezultat šalje u red prema UART tasku.

8. UART protokol između STM32 i ESP32

Dvije ploče komuniciraju serijski (UART, 115200 bps, 8N1) jednostavnim **binarnim paketnim protokolom**. Svaki paket počinje s dva «start» bajta (0xAA 0xBB) i završava s dva «end» bajta (0xCC 0xDD), a između ima bajt tipa i korisni teret. Start/end markeri omogućuju prijemniku da se «uhvati» na početak paketa čak i ako propusti koji bajt.

Tip	Naziv	Duljina	Sadržaj
0x02	2D kut	8 B	AZ(2) + jakost(1)
0x03	3D kut	10 B	AZ(2) + EL(2) + jakost(1)
0x04	Sirovi capture	~8 KB	NCH + N + NCH×N×uint16 (big-endian)

Tablica 6. Tipovi paketa (uart_driver.c na obje strane). Sustav primarno koristi 0x03.

3D paket (tip 0x03), 10 bajtova:

AA	BB	03	AZ_H	AZ_L	EL_H	EL_L	STR	CC	DD
----	----	----	------	------	------	------	-----	----	----

start start tip azimut(BE) elevacija(BE) jak. end end

AZ, EL = int16 big-endian u desetinkama ° (npr. 0x012C = 300 = 30.0°)

Isječak 21. Raspored 3D paketa. «BE» = big-endian (viši bajt prvi).

Brojevi se šalju kao **desetinke stupnja** u cjelobrojnom tipu int16 (npr. 30.5° → 305) jer je cijele brojeve jednostavnije i pouzdanije prenijeti od decimalnih, a desetinka stupnja je više nego dovoljna preciznost.

9. ESP32: Wi-Fi, web-poslužitelj i 3D vizualizacija

ESP32 je «prezentacijski» dio sustava. Pri pokretanju (app_main) redom inicijalizira: NVS (pohrana za Wi-Fi), Wi-Fi u Access Point modu, HTTP poslužitelj, UART i procesor primljenih paketa.

```
void app_main(void) {
    nvs_flash_init();           /* trajna pohrana (Wi-Fi treba NVS)      */
    wifi_manager_init();        /* podigni vlastitu Wi-Fi mrežu (AP)    */
    web_server_init();          /* HTTP + WebSocket na portu 80         */
    uart_driver_init();         /* UART1: RX=GPI016, TX=GPI017, 115200 */
    sound_loc_processor_init(); /* pokreni rx_task (parser paketa)      */
}
```

Isječak 22. Redoslijed inicijalizacije na ESP32 (main.c).

9.1 Wi-Fi Access Point

ESP32 ne spaja se na postojeću mrežu, nego **stvara vlastitu**. To znači da sustav radi samostalno, bez routera. Korisnik se telefonom ili laptopom spoji na tu mrežu i otvori adresu poslužitelja.

Parametar	Vrijednost
SSID (ime mreže)	SoundLocalization
Lozinka	soundloc123
IP poslužitelja	192.168.4.1
Maks. klijenata	4

Tablica 7. Wi-Fi AP postavke (wifi_manager.c). U pregledniku otvoriti <http://192.168.4.1>

9.2 Parser paketa — konačni automat (state machine)

UART podaci stižu kao **tok bajtova**, bez jasnih granica. Da iz toka izvuče pakete, rx_task koristi **konačni automat**: za svaki bajt prelazi u sljedeće stanje (čekam SOF1 → SOF2 → tip → AZ_H → ... → EOF). Tek kad uredno stigne do kraja s ispravnim end-markerom, paket se smatra valjanim i proslijeđuje vizualizaciji.

```
switch (state) {
    case S_SOF1: if (b==0xAA) state=S_SOF2; break; /* čekaj start */
    case S_SOF2: state=(b==0xBB)?S_TYPE:S_SOF1; break;
    case S_TYPE: pkt_type=b;
                 state=(b==0x03)?S_AZ_H:(b==0x04)?S_RAW_NCH:S_SOF1; break;
    case S_AZ_H: az_h=b; state=S_AZ_L; break;
    ...
    case S_EOF2: if (b==0xDD) { /* paket potpun i ispravan */
                 int16_t az=(az_h<<8)|az_l;
                 web_server_send_data(az/10.0f, e1/10.0f, str_val);
                 state=S_SOF1; break;
    }
```

Isječak 23. Parser kao konačni automat (sound_loc_processor.c). Otporno na izgubljene/iskrivljene bajtove.

Zašto automat, a ne «pročitaj 10 bajtova»

Ako se izgubi samo jedan bajt, naivno čitanje fiksne duljine zauvijek bi ostalo pomaknuto. Automat se uvijek iznova sinkronizira na start-marker 0xAA 0xBB, pa se nakon greške sam oporavi na sljedećem paketu.

9.3 WebSocket — gurni podatke u preglednik čim stignu

Klasični HTTP traži da preglednik *pita* poslužitelj. Za podatke koji stižu spontano (svaki pljesak) prikladniji je **WebSocket** — trajna veza preko koje poslužitelj može **sam** poslati podatak čim ga ima. Za svaku detekciju ESP32 svim spojenim preglednicima pošalje kratki JSON.

```
/* ESP32 → preglednik, za svaku detekciju: */
{"azimuth":30.0,"polar":20.0,"strength":42}
```

Isječak 24. JSON poruka preko WebSocket-a (*web_server.c*, *web_server_send_data*).

Lista spojenih klijenata čuva se uz **mutex** (uzajamno isključenje) jer joj pristupaju dvije niti — ona koja prima nove veze i ona koja šalje podatke. Bez zaključavanja moglo bi doći do oštećenja liste; mrtve veze (zatvoreni preglednik) automatski se uklanjaju.

9.4 3D vizualizacija (WebGL)

Sama web-stranica ugrađena je kao tekst u firmware ESP32 i poslužuje se na korijenskoj adresi «/». Crta minimalnu 3D scenu pomoću WebGL-a:

- Plava kocka u središtu predstavlja mikrofonski niz (slušatelja).
- Tri prozirne ravnine (XY, XZ, YZ) daju osjećaj prostora i orijentacije.
- Svaka detekcija dodaje **svjetleću kuglicu** na izračunatom smjeru (azimut + elevacija na zamišljenoj sferi), koja kroz 5 s polako izbljedi.
- Mišem/dodirom scena se može slobodno zakretati i zumirati (orbit kamera).

Pretvorba kuta u 3D položaj kuglice

Iz azimuta (a) i elevacije/polara (p) položaj na sferi polumjera 8 računa se kao $x = 8 \cdot \sin(a) \cdot \cos(p)$, $y = 8 \cdot \sin(p)$, $z = -8 \cdot \cos(a) \cdot \cos(p)$. Tako kut iz STM32 izračuna postaje konkretna točka koju oko vidi.

10. Cjeloviti primjer: od pljeska do točke na ekranu

Spojimo sve u jedan slijed događaja za **jedan pljesak** iz smjera azimut +30°, elevacija +20°:

- 1 t = 0 ms.** Zvučni val stiže na mikrofone u malo različitim trenucima (M2 i M3 ranije/kasnije od M1, ovisno o smjeru).
- 2 Kontinuirano.** TIM8 okida ADC svakih 15.6 μ s; ADC redom čita M1–M4; DMA puni *adc_buffer* bez procesora.
- 3 Svaki 16 ms.** DMA javi HT ili TC prekid → ISR pošalje događaj u red → *ACQ_Task* kopira svježju polovicu u snapshot i javi *FFT_Task*-u.
- 4 FFT_Task.** Sastavi klizni prozor (2048), pozove *LOC3D_Process*: nađe energetski vrh, izdvoji kanale (DC + Hann), provjeri RMS prag.
- 5 GCC-PHAT × 3.** Izmjeri kašnjenja $\tau_{12} \approx -274 \mu$ s, $\tau_{13} \approx -137 \mu$ s, $\tau_{14} \approx -198 \mu$ s (nakon korekcije kanalnog offseta).
- 6 Geometrija.** $u = M_{\text{geom}} \cdot \tau$, pa $s = -u/|u| \approx (0.814, 0.470, 0.342)$.
- 7 Kutovi.** azimut = $\text{atan2}(0.470, 0.814) = 30.0^\circ$, elevacija = $\text{asin}(0.342) = 20.0^\circ$, jakost ≈ 42 .
- 8 UART.** *UART_Task* pošalje 10-bajtni paket [AA BB 03 01 2C 00 C8 2A CC DD] na ESP32.
- 9 ESP32.** *rx_task* automatom sklopi paket, izračuna $30.0^\circ/20.0^\circ$, pošalje JSON preko WebSocket-a.
- 10 Preglednik.** Doda svjetleću kuglicu na (x,y,z) koji odgovara $30^\circ/20^\circ$; HUD ispiše «Azimuth: 30.0° | Polar: 20.0° | Strength: 42».
- 11 Cooldown.** STM32 idućih ~304 ms ignorira nove detekcije da isti pljesak ne ispali više točaka.

0x012C = 300 = 30.0° (azimut), 0x00C8 = 200 = 20.0° (elevacija), 0x2A = 42 (jakost) — bajtovi u paketu iz koraka 8.

11. Pojmovnik za početnike

Pojam	Objašnjenje
ADC	Analog-to-Digital Converter — pretvara napon u broj (ovdje 12-bitni, 0–4095).
DMA	Direct Memory Access — sklop koji premješta podatke u/iz memorije bez procesora.
ISR	Interrupt Service Routine — kratka rutina koja se izvrši na hardverski prekid.
RTOS	Real-Time OS — sustav koji dijeli procesor na taskove s prioritetima.
Task	Neovisna nit izvođenja s vlastitim stogom; vrti se u petlji.
Stack	Stog — memorija za lokalne varijable i povratne adrese jednog taska.
Heap	Gomila — zajednička memorija iz koje RTOS alocira objekte.
Queue	Red čekanja — sigurna cijev za predaju podataka među taskovima/ISR-om.
TDOA	Time Difference Of Arrival — razlika vremena dolaska zvuka na dva mikrofona.
Korelacija	Mjera sličnosti dvaju signala u ovisnosti o njihovom međusobnom pomaku.
GCC-PHAT	Izoštrena križna korelacija (samo faza) za precizno mjerenje TDOA.
FFT	Fast Fourier Transform — brzi prelazak signala u frekvencijsku domenu.
DC offset	Istosmjerna (srednja) razina signala; uklanja se prije korelacije.
Hann prozor	Glatko «zvono» kojim se množi signal da se smanje FFT artefakti.
RMS	Root Mean Square — efektivna amplituda (mjera glasnoće) signala.
Azimut	Vodoravni kut smjera (lijevo/desno/naprijed/nazad).
Elevacija	Okomiti kut smjera (gore/dolje).
Interleaved	Isprepleteni raspored: uzorci kanala poredani M1 M2 M3 M4 M1 M2...
Big-endian	Zapis broja s višim bajtom prvim (npr. 300 = 0x01 0x2C).
WebSocket	Trajna dvosmjerna veza preglednik–poslužitelj za «push» podatke.
Access Point	Način rada u kojem ESP32 sam stvara Wi-Fi mrežu.

Kraj dokumenta. Svi isječci koda i konstante preuzeti su izravno iz izvornog koda projekta (mape «Sound Localization» i «ESP32_Visualization»).